

SweetStoryLab

Architecture and Development Guide SweetStoryLab Engineering

v1.3 — Operational Audit & Weekly Performance Report Design Release (updated from v1.1)

This revision was produced from a direct, verified audit of the live repository (github.com/sweetlifeofrosee/sweetstorylab, main branch, fetched and inspected file-by-file — not inferred from prior documentation) plus the finalized architecture for v1.3's first operational feature, the Weekly Performance Report. Every claim below marked **NEW since v1.1** was confirmed against actual source files, not assumed.

1. Executive Summary

What SweetStoryLab Is

SweetStoryLab is a reusable AI storytelling platform, not a single automated Facebook page. It is a Content Factory: one shared codebase capable of generating, assembling, and publishing short-form video content across multiple storytelling “brands” (also called “Universes”) — Horror Lab and Mystery Lab today, with Bible Stories, History Files, Animal Stories, and others planned.

The defining architectural rule of the platform is this: the engine must never know what genre it is generating. Every brand supplies its own configuration, prompts, branding, voice mapping, fallback content, and Facebook credentials. The engine (core/) only ever knows about pipeline stages — generate a story, render segments, generate narration, assemble a video, publish, log — never about horror, mystery, or any other genre's content. This rule was re-verified during this audit and still holds without exception.

Goals of the Project

- **Modularity** — each responsibility is isolated behind its own interface.
- **Maintainability** — a bug fix or improvement to core mechanics benefits every brand simultaneously.
- **Scalability** — designed against “will this still work with 50 brands?”, not just the two currently in scope.
- **Provider independence** — every external service sits behind an abstract interface, swappable without touching brand code.
- **Configuration over hardcoding** — brand identity lives entirely in data, never hardcoded into the engine.

What v1.0 and v1.1 Accomplished

v1.0 was the extraction and validation of Horror Lab out of a monolithic script into the modular core/ + brands/horror_lab/ architecture. v1.1 added Mystery Lab as a second, fully independent brand; a shared adaptive text-layout system; per-brand visual identity via config; a reworked subtitle treatment; and a --count developer testing flag. Full detail is preserved in §2 (Phases 0–17), unchanged from the v1.1 document.

What This Update (v1.3) Adds

Since v1.1, two categories of change are reflected in this revision:

- **Real, shipped architecture the v1.1 guide never documented**, found via this audit: an optional, additive per-brand deduplication/editorial-memory mechanism in `core/story/engine.py`, used today by Mystery Lab only; a new `content.dedup_module` config field; brand-specific SQLite log files via a new `--db-path` flag; and formalized CLI exit-code semantics. See §3, §7, and the new §10.9.
- **The finalized, approved-but-not-yet-implemented architecture for the Weekly Performance Report**, SweetStoryLab's first operational (non-generation) feature — a read-only, repository-first weekly summary of how each brand performed, deliberately isolated from the generation/posting pipeline. See the new §14.

Note: This audit also surfaced a real gap directly relevant to the Weekly Performance Report design: the per-post operational log (status, errors, timestamps) is **not currently persisted in the repository** for either brand — it exists only as a 30-day-retention GitHub Actions artifact. This is documented honestly in §7 and §12 rather than assumed away. See §14.5 for how the Weekly Performance Report design accounts for it.

2. Project Timeline

Phases 0–17, covering the original Horror Lab extraction, the v1.0 release, and the v1.1 Mystery Lab and rendering work, are unchanged from the v1.1 document and are not reproduced in full here for length — see the v1.1 guide for the complete phase-by-phase record. Two phases have been added since, reflecting the work captured in this revision.

Phase	Summary (unchanged from v1.1 — see that document for full detail)
Phase 0	The starting point. A single working script, <code>post.py</code> , already live in production, automating Horror Lab's Facebook Reels.
Phase 1	Vision and platform framing — reframed from automating one brand to building a multi-brand platform.
Phase 2	Architectural design (no code). The core/brand boundary defined on paper.
Phase 3	First extraction pass, later found to contain fabricated functionality.
Phase 4	Real-file reconciliation against the actual <code>post.py</code> ; produced <code>ASSUMPTION_AUDIT.md</code> .
Phase 5	Two deliberate architectural decisions: opaque <code>voice_profile</code> ; core/brand split for fallback resilience.
Phase 6	Architecture sign-off review; found recurring silent-default-to-Horror-Lab pattern.
Phase 7	v1.0 release audit; found a release-blocking dynamic-import bug.
Phase 8	Live validation with real credentials; surfaced three further real bugs.
Phase 9	v1.0 tagged — first successful real, live Facebook Reel publish.
Phase 10	Mystery Lab brand identity design (Brand Manifesto, pre-code).
Phase 11	Mystery Lab implementation — zero core/ changes required, confirmed by direct verification.
Phase 12	Hero title renderer redesign — adaptive, cinematic hero title.
Phase 13	Question slide redesigned as the emotional climax.
Phase 14	Shared adaptive text-layout extraction (<code>core/renderers/text_layout.py</code>).
Phase 15	Subtitle treatment overhaul; real empirical FFmpeg/libass calibration (§10.7–10.8).
Phase 16	Per-brand visual identity generalized via config, twice more (<code>question.max_font_size</code> , <code>question.text_color</code>).
Phase 17	--count developer testing feature added to the CLI.

Phase 18 — Editorial memory audit and documentation (this revision)

Not new engineering work in itself, but the first point at which the platform's actual deployed state was directly audited against the live repository rather than against prior documentation. This surfaced an optional per-brand subject-deduplication mechanism (`core/story/engine.py`'s `dedup_checker` parameter, and Mystery Lab's `brands/mystery_lab/dedup.py`) that had shipped and was running in production without ever being written up. The mechanism itself follows the platform's established pattern exactly — additive, opt-in, complete no-op for brands that don't use it — but its existence had drifted out of the documentation. See §3 and §10.9 for what the audit found.

Phase 19 — Weekly Performance Report: design and freeze

SweetStoryLab's first Operations & Analytics-phase feature, and its first feature explicitly framed as non-generation work: a weekly, cross-brand performance summary. Designed and iterated to a frozen v1.3

specification across several rounds of explicit review (repository-first stance; CSV-first platform-data adapter with a swap-ready interface for a future live API; a recommended `story_id` join key; separation of Operational Health from Editorial Intelligence; and a standing “descriptive, not prescriptive” principle). See the new §14 for the full approved design.

3. Architecture Overview

The Core Principle

Unchanged from v1.0/v1.1, and re-verified during this audit by direct inspection of `core/`: it remains genre-agnostic. It has no knowledge of Horror Lab, Mystery Lab, or any brand's content.

Module Responsibilities

All v1.1 module descriptions remain accurate. One module gained new, documented behavior since v1.1, and one brand-owned module is entirely new.

- **core/story/engine.py** (updated since v1.1) — still owns the generic “try generation, validate, fall back” mechanism and generic segment-validity checking. **New:** `generate_story()` now accepts an optional `dedup_checker` argument. When absent (Horror Lab today), behavior is byte-identical to before this capability existed. When present (Mystery Lab), a successfully parsed, structurally valid Story is additionally checked via `dedup_checker.is_acceptable(story)`; on rejection, the story is regenerated (bounded retries) with a retry hint appended to the prompt.
- **brands/mystery_lab/dedup.py** (new since v1.1) — `MysteryLabSubjectDeduplicator`. Brand-owned, not shared with core or with Horror Lab. Tracks accepted stories' `primary_subject` values in a flat JSON file (`brands/mystery_lab/used_subjects.json`), case-insensitive and leading-“the”-normalized, and rejects a newly generated story if its subject has already been used. No curated topic database — it only ever knows about subjects the model itself has generated and had accepted.
- **core/config/loader.py** (updated since v1.1) — `BrandConfig` gained one new, optional field: `dedup_module` (dotted path to a brand's deduplication checker, or `None`). Not required — Horror Lab's `config.yaml` simply omits it.
- **core/pipeline/cli.py** (updated since v1.1) — gained a `--db-path` argument (default `posts.db`), now passed explicitly and differently per brand by each workflow file (`horror_log.db`, `mystery_log.db`). Exit-code semantics are now explicit in code comments: 0 = completed (published or valid dry run), 1 = crashed before completion, 2 = video generated but the real publish attempt failed.

`core/renderers/text_layout.py`, `narration_scene.py`, `question_slide.py`, `core/providers/*` (llm, image, tts, music, publish), `core/video/assembler.py`, and `core/storage/log_store.py` all match their v1.1 descriptions exactly — confirmed by direct inspection, no changes found.

4. Design Philosophy

All v1.0/v1.1 principles hold unchanged (separation of concerns, dependency direction, brand isolation, provider abstraction, configuration-driven design). The dedup mechanism found in this audit is a fresh, clean instance of the same “core owns generic mechanism, brand owns content” split already established for fallback resilience in Phase 5: `core/story/engine.py` knows only “call `is_acceptable`, call `retry_hint` on rejection”; it has no idea this means “check a JSON file” — that’s entirely `brands/mystery_lab/dedup.py`’s concern. The pattern transferred without modification, which is itself a small piece of evidence the boundary is well-drawn.

The existing “When Is Sharing Code the Right Call?” design tension from v1.1 (§4 of that document) is unchanged and still governs: brand-identity duplication (Horror Lab’s and Mystery Lab’s parsers) stays separate; generic mechanism needed by core itself gets shared. The dedup mechanism is a third example of the latter — it lives in `core/story/engine.py` as a generic hook, with only the brand-specific storage format living in brand code.

5. Folder Structure

Verified directly against the live repository during this audit (not reconstructed from prior docs). Root-level legacy files (post.py, index.html, sounds/, callback.html, privacy-policy.html, terms-of-service.html, favicon assets, a stray tiktok verification .txt) remain present but are not part of the current architecture — they predate the core/brands extraction and are omitted below for clarity, exactly as they were genuinely confirmed to still exist but unused by the current pipeline.

```
sweetystorylab/
|-- core/
| |-- config/
| | |-- loader.py # UPDATED -- dedup_module field (optional)
| | |-- platform.py
| |-- story/
| | |-- schema.py # Story.primary_subject (added pre-audit, undocumented until now)
| | |-- parser_base.py
| | |-- fallback_base.py
| | |-- engine.py # UPDATED -- optional dedup_checker parameter
| |-- renderers/
| | |-- registry.py
| | |-- text_layout.py
| | |-- narration_scene.py
| | |-- question_slide.py
| |-- providers/
| | |-- llm/ (groq_provider.py)
| | |-- image/ (pollinations_provider.py)
| | |-- tts/ (edge_tts_provider.py, elevenlabs_provider.py, orchestrator.py)
| | |-- music/ (local_file_provider.py)
| | |-- publish/ (facebook_provider.py)
| |-- video/
| | |-- assembler.py
| |-- storage/
| | |-- log_store.py # schema unchanged; now used via per-brand --db-path files
| |-- pipeline/
| |-- run.py # run_brand(), run_brand_batch()
| |-- cli.py # UPDATED -- --db-path flag; exit codes 0/1/2 formalized
|
|-- brands/
| |-- horror_lab/
| | |-- config.yaml # no dedup_module -- dedup is fully opt-in
| | |-- prompts/
| | |-- parser.py
| | |-- fallback.py
| | |-- assets/
| |-- mystery_lab/
| | |-- config.yaml # NEW field: content.dedup_module
| | |-- prompts/
| | |-- parser.py
| | |-- fallback.py
| | |-- dedup.py # NEW since v1.1 -- MysteryLabSubjectDeduplicator
| | |-- used_subjects.json # NEW since v1.1 -- committed editorial memory (subjects only)
| | |-- assets/
|
|-- docs/ # NEW since v1.1 -- this guide now lives in-repo
| |-- SweetyStoryLab_v1.1_Architecture_and_Development_Guide.pdf
| |-- architecture.md # pre-v1.0 planning brief; superseded, kept for history
|
|-- .github/workflows/
```

```
|-- post.yml # Horror Lab -- no persistence step, log is artifact-only  
`-- post_mystery.yml # Mystery Lab -- UPDATED: contents:write permission +  
# dedicated step to commit used_subjects.json back to the repo
```

Note: docs/architecture.md is an earlier, pre-v1.0 planning brief still referring to the second brand as “Mystery Archive.” It predates and is fully superseded by this guide. Recommend either deleting it or adding a one-line pointer to this document to avoid confusing future readers.

6. End-to-End Pipeline Walkthrough

Unchanged in structure from v1.1 (that document's §6 remains accurate step-by-step) with the dedup hook now confirmed as a real, additional step for brands that opt in:

- Step 3 (story generation) now optionally routes through the dedup_checker described in §3 above, when the brand's config.yaml sets content.dedup_module. For Horror Lab, this step is a complete no-op, confirmed by its config.yaml containing no such key.
- Step 6 (per-segment rendering) resolves question_max_font_size and question_text_color from brand config into RenderContext, unchanged from v1.1.
- The --count N batch mode (Phase 17) is unchanged: routes to run_brand_batch(), forces dry-run, saves each story's full output under output/<brand>/<timestamp>/story_NN/.

A new operational detail, confirmed via the actual workflow files: after a normal (non-batch) Mystery Lab run completes, post_mystery.yml checks whether brands/mystery_lab/used_subjects.json changed and, if so, commits and pushes it back to the repository under the github-actions[bot] identity. Horror Lab's workflow has no equivalent step, matching its brand having no dedup module.

7. Configuration Reference

All v1.1 fields remain accurate. One field is new since v1.1:

Field	Owner	Consumed by
content.dedup_module (NEW)	Brand (optional)	core/config/loader.py reads it into BrandConfig.dedup_module (Optional[str]). run.py dynamically imports it (when present) and passes the resulting checker into core.story.engine.generate_story() as dedup_checker. Only Mystery Lab's config.yaml sets this today; Horror Lab's config.yaml omits it entirely, which is a fully supported, zero-behavior-change state.

Current per-brand values, updated:

Field	Horror Lab	Mystery Lab
question.max_font_size	110pt	117pt
question.text_color	White (255,255,255)	Gold (198,156,84)
branding.watermark_text	"SweetyStoryLab"	"Mystery Lab"
content.dedup_module (NEW)	not set	brands.mystery_lab.dedup.MysteryLabSubjectDeduplicator
llm.temperature	0.95	0.85

Operational log persistence — current state (confirmed, not assumed)

Note: Both post.yml and post_mystery.yml invoke the pipeline with a brand-specific --db-path (horror_log.db / mystery_log.db), and both upload that file only via actions/upload-artifact@v4 with a 30-day retention window. **Neither workflow commits the SQLite log back to the repository.** This means, as of this audit, the per-post operational record (status, video_id, error, timestamp) is not durably stored anywhere longer than 30 days. Only Mystery Lab's used_subjects.json (subjects covered, not full post outcomes) is committed to the repo. This is carried forward into §14 (Weekly Performance Report) as a known, explicitly documented gap rather than an assumption the design silently depends on.

8. Providers

Unchanged from v1.1, confirmed by direct inspection of `core/providers/`: LLMProvider (GroqProvider), ImageProvider (PollinationsProvider), TTSPProvider (EdgeTTSPProvider, ElevenLabsProvider, plus an `orchestrator.py` that wasn't itemized in the v1.1 text but implements the existing TTS fallback behavior described there), MusicProvider (LocalFileProvider), PublishProvider (FacebookProvider) — each with exactly one production implementation, each swappable without touching brand code. No new provider types were added since v1.1.

9. Brand Development Guide

Steps are unchanged from v1.1. One addition, informed directly by the dedup mechanism found in this audit: if a new brand wants subject/topic-level deduplication across its own history, it can opt in the same way Mystery Lab does — supply a dedup checker exposing `is_acceptable(story)` and `retry_hint(story)`, point `content.dedup_module` at it in `config.yaml`, and `core/story/engine.py` handles the retry loop generically. Brands that don't need this simply omit the field, exactly as Horror Lab does today.

10. Major Debugging Journey

Sections 10.1–10.8 are unchanged from the v1.1 document (dynamic import path failure, Windows FFmpeg subtitle path parsing, asset path ownership regression, local vs. CI testing, Facebook authentication, runtime validation gaps, the FFmpeg original_size empirical-calibration finding, and the two-unbounded-input filtergraph hang) — see that document for full detail. One new entry follows, from this audit.

10.9 — Documentation drift: a shipped mechanism the guide never recorded

Finding: core/story/engine.py's dedup_checker parameter, brands/mystery_lab/dedup.py, the content.dedup_module config field, used_subjects.json, and the git-commit-back step in post_mystery.yml were all present and functioning in the live repository, but none appeared anywhere in the v1.1 guide.

Root cause: the work was real, deliberate, and well-engineered (it follows the platform's own established core/brand-isolation pattern precisely) — it simply wasn't captured back into this document when it shipped, unlike every other architectural change in Phases 0–17.

How it was caught: not by re-reading prior documentation, but by fetching and inspecting the actual repository directly — first via GitHub's web view, then (after a robots.txt restriction on that path) via the GitHub REST API and a full repository archive, read file-by-file rather than assumed from folder names.

Why this matters architecturally: this is a direct instance of the same lesson the v1.1 guide already recorded twice (“claiming a fix was applied is not the same as verifying it was”), applied one level up — to documentation itself, not just code. A guide that isn't re-verified against the real repository can silently drift from what's actually running, exactly the way an unmeasured code fix can silently not work. See §13 for the lesson entry.

11. Important Design Decisions

All v1.0/v1.1 decisions stand unchanged. One more is recorded from this revision's work:

- **Chosen:** treat the Weekly Performance Report as strictly read-only and repository-first, with platform analytics as optional enrichment behind a swappable adapter, rather than building any live platform-API integration in its first implementation. **Rejected alternative:** implementing Graph API integration immediately. **Reason:** validating the reporting architecture (aggregation, scoring, rendering) against a known-stable input (an existing Meta CSV export already in use) de-risks the harder problem (rendering and scoring logic) before adding the variable one (live API integration, credentials, rate limits). The adapter interface (§14.3) means this is a deliberate sequencing choice, not a permanent architectural limitation.
- **Chosen:** the Weekly Performance Report is explicitly descriptive, not prescriptive — it has no code path that writes to, or is read by, any part of the generation or posting pipeline. **Reason:** keeps the platform's core/brand behavioral boundary (§3, §4) fully intact; a reporting feature is exactly the kind of addition that could quietly grow into a control surface over time if this weren't stated as an explicit, standing constraint rather than an implicit assumption. See §14.2.

12. Future Improvements (Not Yet Implemented)

Updated from the v1.1 list. Items already completed are removed; new items from this audit and from the Weekly Performance Report design are added.

- A generic, JSON-based StoryParser implementation, offered as an adoptable option.
- A real segment_template enforcement mechanism — still just documentation of intended shape today (confirmed unchanged: core/config/loader.py loads it but does not validate generated segments against it beyond the existing generic content_fields check in engine.py).
- Multi-platform publishing (YouTube Shorts, TikTok).
- Cross-brand rate-limit coordination for shared platform-level Groq/ElevenLabs accounts — still not needed; two brands running independently hasn't produced a real coordination need.
- A per-brand or fleet-wide dashboard — the first real consumer of brand.emoji / brand.parent.
- Reconsidering per-brand GitHub Actions workflow duplication.
- A generic Horror Lab/Mystery Lab parser abstraction — deliberately still deferred.
- A third brand.
- **NEW:** durable, repo- or otherwise long-term-persisted operational logging. Today's per-post status/error/timestamp history exists only as a 30-day GitHub Actions artifact for both brands (§7). Extending the existing used_subjects.json commit-back pattern to a compact operational log (or an equivalent) would close this gap and strengthen the Weekly Performance Report's Operational Health section (§14.4) beyond its current per-run, non-cumulative view.
- **NEW:** category/theme/hook capture in the Story schema and both brands' parsers. Confirmed via direct inspection of core/story/schema.py and both parser.py files: no such fields exist today, only primary_subject (Mystery Lab only). The Weekly Performance Report's Editorial Intelligence section (§14.4) documents best-performing categories/themes/hooks as designed capabilities that currently have no data source — this is the concrete follow-up work that would populate them.
- **NEW (recommended in the Weekly Performance Report design, §14.3):** a lightweight story_id field, assigned per published story, to support joins between editorial history and platform analytics that don't rely solely on brand + publish date — needed once multiple posts per day, reposts, retries, or additional platforms are in play. Confirmed: no such field exists anywhere in the schema today.
- **NEW:** swap the Weekly Performance Report's CSV-based platform-data source for a live Graph API adapter, once the reporting architecture itself has been validated in production (§14.3).

13. Lessons Learned

All prior lessons stand. One new lesson from this revision:

“A guide is a claim about the codebase, and claims need re-verification just like fixes do.” The dedup mechanism found in §10.9 wasn't a mistake in the code — it was correct, working, and well-isolated. The gap was purely documentary: real shipped behavior that the architecture guide didn't reflect, discovered only because this revision was produced by auditing the live repository directly rather than extending the previous document's own account of itself. This is the same evidence standard the v1.1 guide already applied to code (“claiming a fix was applied is not the same as verifying it was”), now applied to documentation: a section describing the architecture is a claim, and claims drift from reality unless someone periodically checks the claim against the thing itself, not against the last version of the claim.

14. Weekly Performance Report — Approved Architecture (v1.3, Not Yet Implemented)

SweetyStoryLab's first Operations & Analytics-phase feature. This section documents the design as reviewed and frozen across multiple rounds; it has not yet been built. Status: **approved, ready for implementation**.

14.1 Goal & Non-Goals

Goal: produce a weekly Markdown report summarizing how Horror Lab and Mystery Lab (and any future brands) performed, combining platform metrics with repo-side editorial signals into a single cross-brand view.

Non-goals: no changes to prompts, generation logic, or posting behavior; no changes to the existing production workflows for Horror Lab / Mystery Lab; read-only with respect to content pipelines.

14.2 Design Principles

- **Repository-first.** The repo-side editorial history is the source of truth the report can always build from. Platform metrics are supplementary enrichment — valuable when present, never a dependency. The report generates every week regardless of whether any platform data source is reachable.
- **Descriptive, not prescriptive.** The report summarizes historical performance. It does not influence generation, prompts, scheduling, or topic selection, and nothing in this pipeline writes back to any file the generation or posting pipelines read from. If AI-assisted recommendations are built later, they consume this reporting layer as an input — they do not modify it.
- **Fail-soft per source.** If the platform export is missing, stale, or malformed, the report still generates with a “data unavailable” note for that section rather than failing the whole run.
- **Source-agnostic aggregation.** Aggregation and rendering logic never change when the platform-data source is later swapped (§14.3).
- **No shared state with generation.** This workflow never writes to any file the generation/posting pipelines read from, and needs no new API credentials for its first implementation.

14.3 Data Sources

Source	Role	Access method	Required?
Repo-side editorial history (Mystery Lab's used_subjects.json today; Horror Lab has no equivalent yet)	Primary — source of truth	Direct read from repo JSON/log files	Yes
Platform data — v1: exported CSVs (e.g. the Meta export already in use)	Supplementary enrichment	Manually-dropped or committed CSV files, fixed data/exports/ path	No

Data source abstraction (for the later API swap). Platform data is accessed only through a small adapter interface, so the report can be validated on CSVs now and swapped to a live Graph API later without touching aggregation or rendering:

```
scripts/report/platform_source/  
base.py # get_metrics(brand, week) -> normalized dict, or None if unavailable  
csv_source.py # v1: reads the Meta-style CSV export  
graph_api_source.py # v2 (future, not built yet): same interface, calls Graph API instead
```

aggregate.py and render_report.py only ever talk to the normalized dict returned by get_metrics() — swapping sources later is a one-line change (which adapter gets instantiated), with no changes to scoring or rendering logic. The normalized dict includes an optional story_id field (see below) alongside brand/date.

Recommended field: story_id (future compatibility, not required for v1)

Joining repo editorial history to platform analytics by brand + publish date works for today's one-post-per-day model, but breaks down once any of the following becomes true: multiple posts per day, reposts, retries after a failed publish, or analytics from additional platforms with their own timestamps. The reporting model should reserve a lightweight story_id field — a stable, unique identifier per published story — as the preferred join key wherever available, with brand + publish date remaining the fallback. No production pipeline change is required now; when/if a story_id is introduced upstream, the aggregation layer should prefer it automatically. Format is intentionally unspecified — whatever's cheapest to generate where stories are currently created (a hash, a timestamp-based slug, a UUID) is fine.

14.4 Report Structure

Operational Health and Editorial Intelligence are deliberately separate top-level sections, each with per-brand subsections — not a merged “insights” block. One tells you whether the pipeline ran cleanly; the other tells you what the content actually was and how it's trending.

```
# Weekly Performance Report -- Week of {date}

## Summary
- Cross-brand table: score, WoW change, top metric mover per brand
- Note if any brand's score is operational-only this week (platform data unavailable)

## Operational Health
### Horror Lab / ### Mystery Lab
- Posting consistency (planned vs. actual), pipeline run status
- Generation/publishing errors or skips

## Editorial Intelligence
### Horror Lab / ### Mystery Lab
- Best-performing categories, best-performing themes, top hooks
- Category frequency, subjects covered
- Editorial observations

## Audience & Engagement
### Horror Lab / ### Mystery Lab
- Views, watch time, follower delta, likes, comments, shares, retention
- Notable posts (best & worst performing), when platform data present

## Trends
- 4-week rolling chart data

## Data Notes
- Any sources that failed to fetch this week, flagged explicitly
```

Weekly Performance Score — weighted composite

Pillar	Example metrics	Default weight
Audience	Views, watch time, follower delta	40%
Engagement	Likes, comments, shares, retention %	40%

Pillar	Example metrics	Default weight
Operational Health	Posting consistency, pipeline run status, generation/publishing errors	20%

Each pillar is normalized to a 0–100 scale relative to that brand's trailing 4-week average before weighting. Weights live in `scripts/report/weights.yaml`. **Degraded mode:** if platform data is unavailable for a brand that week, Audience and Engagement are omitted (not zeroed) and the composite score is computed from Operational Health alone, labeled “operational-only score, platform data unavailable this week.”

Editorial Intelligence — content detail

- Best-performing categories, best-performing themes, top hooks — ranked by performance where platform data is available, otherwise surfaced as frequency/representative samples for editorial review.
- Category frequency — distribution this week vs. the trailing 4-week distribution.
- Subjects covered — specific topics/subjects touched on, to spot repetition or gaps at a glance.
- Editorial observations — short freeform, pattern-level notes generated from the aggregation step.

Note: As confirmed in §7 and §12, no category/theme/hook fields exist in the Story schema or either brand's parser today, and only Mystery Lab captures `primary_subject`. This section of the report is designed and approved, but several of its fields have no data source yet until that schema work (§12) is done. This is stated here explicitly so implementation doesn't silently stub these out or fabricate placeholder data.

Operational Health — pipeline detail

- Posting consistency — planned vs. actual posts this week, gaps or skips.
- Pipeline health — whether scheduled generation/publishing runs completed, partial-failure counts.
- Generation/publishing errors — summarized counts from automation logs, not full stack traces.

Note: As documented in §7, today's per-post operational log is artifact-only (30-day retention via `actions/upload-artifact`), not committed to the repo, for both brands. Per the decision recorded with this revision, the report design and this guide document that current state as-is rather than silently assuming durable repo storage; closing this gap is tracked as a Future Improvement (§12), not a blocking requirement for the report's first implementation. In its absence, this section can only reliably reflect the current run's outcome, not a rolling multi-week operational history, until the persistence gap is closed.

14.5 Architecture & Delivery

A new, isolated GitHub Actions workflow, following the same per-brand-workflow pattern already established by `post.yml` / `post_mystery.yml`, but entirely separate from both:

```
.github/workflows/weekly-report.yml (new -- cron, e.g. Monday 06:00 UTC)
-> scripts/report/load_editorial_history.py (reads repo-side logs/history -- always runs)
-> scripts/report/platform_source/csv_source.py (reads CSV export if present -- optional)
-> scripts/report/aggregate.py (joins + computes composite scores)
-> scripts/report/render_report.py (renders Markdown from template)
-> commits reports/weekly/YYYY-Www.md to the repo
```

Rendered Markdown is committed to `reports/weekly/YYYY-Www.md`, with an optional `reports/weekly/latest.md` for easy reference. Uses its own read-only credentials scope, separate from either posting workflow's secrets, and shares no state with the generation/posting pipelines — consistent with §3's core principle and §11's descriptive-not-prescriptive decision.

14.6 Open Items Before Implementation

- Exact schema/location for a Horror Lab editorial-history equivalent (it currently has none — only Mystery Lab persists anything today, and only subjects).
- Sample column headers from the Meta CSV export, to build `csv_source.py`'s real column mapping.
- Where the CSV export file lands (manual drop vs. an existing process) and whether `csv_source.py` needs to tolerate “file not present yet.”
- Week boundary and timezone for the report window.